ESCUELA ESPECIALIZADA EN INGENIERÍA ITCA-FEPADE, MEGATEC, LA UNIÓN
ESCUELA DE INGENIERÍA DE COMPUTACIÓN
TÉCNICO EN INGENIERÍA DE DESARROLLO DE SOFTWARE

Desarrollo de Aplicaciones Móviles Tercer Examen Práctico
Facilitador: Ing. Eduardo Vásquez
Estudiante: _____ **Fecha:** _____/_____/_____
Estudiante: _____
Estudiante: _____
Estudiante: _____
Grupo: _____

Indicaciones generales:

- El examen se realizará con los mismos integrantes que conforman cada equipo del proyecto de cátedra, no se reorganizarán equipos para esta evaluación.
- Cada equipo presentará un único proyecto Flutter. Todos los integrantes deberán realizar una contribución técnica identificable y ser capaces de explicar su trabajo.
- Se podrán usar paquetes como http o dio para las peticiones, provider o mecanismos equivalentes para el estado y shared_preferences para la persistencia.
- La información del catálogo debe obtenerse mediante solicitudes HTTP reales, no se aceptará reemplazar la integración por una lista de personajes escrita manualmente en el código.
- Se podrá consultar documentación técnica de las API y de Flutter. El uso de otras ayudas deberá respetar las indicaciones institucionales comunicadas por el docente.
- No se requiere una API propia, autenticación, base de datos de servidor ni operaciones POST, PUT, PATCH o DELETE sobre las fuentes públicas seleccionadas.
- Si un miembro del equipo no asiste deberá realizar la evaluación diferida, para la defensa si no se encuentra presente perderá la nota de la defensa, a menos tenga una justificante.
- Únicamente un miembro del equipo subirá el enlace del repositorio, tome en cuenta que deberá elaborar un archivo README.md donde se detallara la descripción de la aplicación, los integrantes y la funcionalidad que posee.

API Asignada: ☐ Los Simpson ☐ Rick and Morty

Evaluación Práctica

Una organización dedicada a divulgar información de series animadas solicita un prototipo móvil para explorar un catálogo de personajes. La aplicación deberá consultar datos actualizados desde una API pública, permitir explorar fichas individuales y conservar una colección de favoritos entre ejecuciones. Al equipo se le asignara una de las dos fuentes de datos indicadas y construirá la misma solución funcional.

Requerimientos funcionales obligatorios

Código	Función	Comportamiento exigido
RF-01	Catálogo de personajes	Pantalla inicial con listado de la primera página obtenida por HTTP. Cada tarjeta mostrará imagen, nombre, estado y un dato adicional propio de la API (especie u ocupación).
RF-02	Detalle y navegación	Al seleccionar un personaje, abrir pantalla de detalle con imagen, nombre, estado y al menos tres datos adicionales disponibles. Ofrecer una navegación clara de regreso.
RF-03	Búsqueda por nombre	Campo de búsqueda que filtre por nombre dentro de los personajes de la página cargada, sin distinguir mayúsculas y minúsculas. Indicar que la búsqueda es local a la página actual.
RF-04	Paginación	Controles Anterior/Siguiente que soliciten páginas diferentes a la API. Mostrar página actual, deshabilitar navegación cuando no exista página previa o siguiente.

Código	Función	Comportamiento exigido
RF-05	Favoritos persistentes	Permitir agregar o quitar personajes favoritos, señalizar el estado visualmente y ofrecer una sección/lista de favoritos. Los identificadores elegidos deben persistir tras cerrar y abrir la app.
RF-06	Estados de la interfaz	Mostrar indicador de carga, estado sin resultados, mensaje comprensible si falla la red o la respuesta y opción de reintentar. No congelar la UI ni mostrar excepciones técnicas al usuario.

. Requerimientos técnicos

- Crear al menos un modelo Dart de personaje con fromJson, tratar correctamente valores nulos, faltantes y estructuras JSON diferentes según la API elegida.
- Separar, como mínimo, pantallas/widgets, modelo y servicio HTTP, no colocar todas las peticiones y el parseo JSON directamente dentro del método build().
- Aplicar gestión de estado para reflejar favoritos y resultados al regresar de una pantalla, puede utilizarse Provider/ChangeNotifier, setState correctamente organizado u otra alternativa explicable.
- Persistir los ID de favoritos con SharedPreferences o alternativa local equivalente, y recuperar esa información al iniciar la aplicación.
- Utilizar URLs HTTPS y mostrar imágenes remotas. En caso de fallo de carga de imagen, utilizar un indicador o sustituto visual.
- Implementar mensajes diferenciados para error de conexión y búsqueda sin resultados, ofrecer reintento ante falla de carga.

Información general

Las URL y campos siguientes se proporcionan para orientar la implementación. Los equipos pueden verificar la documentación oficial antes de codificar.

Alternativa A. Los Simpson

Elemento	Especificación
API y documentación	https://thesimpsonsapi.com/
Listado paginado	GET https://thesimpsonsapi.com/api/characters?page=1
Detalle por ID	GET https://thesimpsonsapi.com/api/characters/1
Campos del personaje	id, name, status, occupation, gender, age, portrait_path, phrases (según disponibilidad).
Respuesta de listado	results y metadatos de paginación: count, pages, next, prev.
Construcción de imagen	https://cdn.thesimpsonsapi.com/500 + portrait_path, ej.: /character/1.webp
Dato adicional sugerido	occupation en catálogo, gender, age y una phrase en detalle cuando estén disponibles.

Alternativa B. Rick and Morty

Elemento	Especificación
API y documentación	https://rickandmortyapi.com/documentation
Listado paginado	GET https://rickandmortyapi.com/api/character?page=1
Detalle por ID	GET https://rickandmortyapi.com/api/character/1
Campos del personaje	id, name, status, species, gender, origin, location, image, episode.
Respuesta de listado	results y metadatos de paginación dentro de info: count, pages, next, prev.
Construcción de imagen	Utilizar directamente la URL proporcionada por el campo image.
Dato adicional sugerido	species en catálogo, gender, origin.name y location.name en detalle.

Uso general de la aplicación

1. Al iniciar la aplicación, se mostrará un indicador de carga mientras se obtiene la información de la API. Una vez completada la consulta, se visualizará el catálogo con los personajes disponibles.
2. El usuario podrá buscar personajes ingresando parte de su nombre. La aplicación filtrará los resultados de la página actual y, si no encuentra coincidencias, mostrará un mensaje informativo.
3. Al seleccionar un personaje, el usuario podrá acceder a una pantalla con su información detallada y tendrá la opción de agregarlo a su lista de favoritos.
4. Desde el catálogo, el usuario podrá navegar entre las diferentes páginas utilizando los controles de paginación para consultar más personajes.
5. El usuario podrá acceder a la sección de Favoritos y visualizar los personajes que haya guardado. Además, deberá comprobar que estos permanecen registrados después de cerrar y volver a abrir la aplicación.
6. Si el dispositivo no cuenta con conexión a internet, la aplicación deberá mostrar un mensaje claro al intentar consultar nuevos personajes y ofrecer una opción para reintentar la conexión.

RÚBRICA DE EVALUACIÓN TERCER EXAMEN PRÁCTICO

Parte 1 de 2: Requerimientos funcionales RF-01 a RF-04

CRITERIO / VALOR	SOBRESALIENTE 100 %	SATISFACTORIO 75 %	EN PROCESO 50 %	NO LOGRADO 0 %
RF-01 · Catálogo de personajes **Máx. 2.00 pts** **Obtenido: _____**	Obtiene por HTTP la primera página real de la API. Cada tarjeta muestra imagen, nombre, estado y especie u ocupación, el catálogo es claro y funcional.	Consulta correctamente la API y muestra los personajes, pero omite un dato requerido o presenta una falla visual menor.	Muestra un catálogo parcial: faltan varios datos, las imágenes o la carga del listado presenta fallas frecuentes.	No presenta el catálogo funcional, utiliza personajes escritos manualmente o no logra obtenerlos de la API.
RF-02 · Detalle y navegación **Máx. 1.00 pts** **Obtenido: _____**	Abre el detalle del personaje seleccionado, presenta imagen, nombre, estado y al menos tres datos adicionales disponibles. Permite regresar sin errores.	Presenta detalle y regreso correctos, pero falta un dato adicional o hay un defecto visual menor.	Abre el detalle de manera parcial: faltan varios datos o la navegación de regreso es inconsistente.	No existe detalle funcional o no se puede abrir el personaje seleccionado.
RF-03 · Búsqueda por nombre **Máx. 1.00 pts** **Obtenido: _____**	Filtra por coincidencias parciales de nombre en la página cargada, sin distinguir mayúsculas, informa el alcance y muestra un estado sin coincidencias.	La búsqueda filtra correctamente, pero omite el aviso de alcance o el mensaje cuando no encuentra coincidencias.	La búsqueda es limitada: exige coincidencia exacta, distingue mayúsculas o entrega resultados incorrectos.	No incorpora una búsqueda funcional por nombre.
RF-04 · Paginación **Máx. 1.00 pts** **Obtenido: _____**	Anterior/Siguiente consulta páginas distintas a la API, muestra el número actual y bloquea correctamente los controles en los límites.	Permite cambiar páginas y muestra el número actual, pero hay un defecto menor en el control de límites o en el estado de botones.	La navegación entre páginas es parcial o inconsistente, faltan controles de límites o no se actualiza el indicador.	No existe paginación funcional o siempre se presenta la misma página.

Niveles de logro: cada criterio se califica con el 100 %, 75 %, 50 % o 0 % de su valor máximo, según la evidencia observada.

RÚBRICA DE EVALUACIÓN · TERCER EXAMEN PRÁCTICO

Parte 2 de 2: Requerimientos funcionales y técnicos

CRITERIO / VALOR	SOBRESALIENTE 100 %	SATISFACTORIO 75 %	EN PROCESO 50 %	NO LOGRADO 0 %
RF-05 · Favoritos persistentes **Máx. 2.00 pts** **Obtenido: _____**	Permite agregar y quitar personajes, refleja el estado visualmente, presenta una lista de favoritos y conserva los ID al cerrar y reabrir la app.	Agrega, elimina y recupera favoritos tras reiniciar, existe una falla menor en los indicadores visuales o en la lista.	Permite marcar favoritos durante la ejecución, pero no los recupera tras reiniciar o carece de una lista funcional.	No implementa favoritos utilizables o la selección no se registra.
RF-06 · Estados de la interfaz **Máx. 1.00 pts** **Obtenido: _____**	Muestra carga, estado sin resultados, errores de red/respuesta comprensibles y opción de reintento, mantiene la interfaz utilizable.	Maneja correctamente la mayoría de los estados, pero omite uno o presenta un mensaje o reintento poco claro.	Solo muestra mensajes genéricos o carga parcial, el usuario no puede recuperar fácilmente una consulta fallida.	No maneja carga ni errores, muestra excepciones técnicas, se bloquea o falla sin orientación.
RT-01 · Consumo HTTP y modelo Dart **Máx. 1.00 pts** **Obtenido: _____**	Consume la API elegida mediante HTTPS, utiliza un modelo con fromJson, interpreta su estructura y contempla campos nulos o faltantes.	Integra HTTPS y fromJson correctamente, presenta fallas menores de mapeo o en el manejo de datos opcionales.	Realiza peticiones pero procesa JSON de forma frágil, sin un modelo suficiente o sin gestionar campos opcionales.	No realiza consumo HTTP real ni dispone de procesamiento útil de las respuestas de la API.
RT-02 · Arquitectura, estado e imágenes **Máx. 1.00 pts** **Obtenido: _____**	Separa pantallas, modelo y servicio HTTP, actualiza correctamente vistas y favoritos, usa imágenes remotas con alternativa ante fallas.	La separación y actualización del estado funcionan, existe una debilidad menor en organización o tratamiento de imágenes.	Concentra demasiada lógica en widgets, presenta estados desactualizados o no controla fallas al cargar imágenes.	La estructura impide el funcionamiento esencial o no mantiene el estado y los recursos visuales requeridos.

REGISTRO DE CALIFICACIÓN

RF-01 a RF-04	RF-05 a RF-06	RT-01 a RT-02	NOTA FINAL
_____ / 5.00	_____ / 3.00	_____ / 2.00	_____ / 10.00

Aplicación de la rúbrica: se evalúa exclusivamente la aplicación Flutter y su funcionamiento técnico. No se incluyen defensa oral.
Cálculo: valor obtenido = valor máximo del criterio × nivel alcanzado. Ejemplo: un criterio de 2.00 pts recibe 2.00, 1.50, 1.00 o 0.00 puntos.